macOS Development Environment Setup & Migration Guide

This document serves as a comprehensive developer reference for configuring a macOS development environment, incorporating your existing Linux workflow (as documented in myLinexEnv) alongside the macOS-specific practices and setup notes from previous configurations.
Line Spacing: 1.25 | Font: Arial

1. Package Management with Homebrew

Unlike Linux which uses apt, macOS utilizes **Homebrew** as its primary package manager for CLI tools and desktop applications (casks).

1.1 Installing Homebrew

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

1.2 Adding Homebrew to PATH (Apple Silicon / Intel)

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

2. Core Utilities & Tooling Migration

Below is the direct mapping of software installation commands from Linux (apt) to macOS (brew).

Category	Linux Command (Ubuntu/Debian)	macOS Command (Homebrew)
Core CLI Tools	sudo apt install curl wget git unzip	brew install curl wget git unzip
Web Server	sudo apt install nginx	brew install nginx
Database Engine	sudo apt install mysql-server mysql-client	brew install mysql
Code Editor	sudo apt install code	brew install --cask

Category	Linux Command (Ubuntu/Debian)	macOS Command (Homebrew)
		visual-studio-code
Database Client	sudo apt install tableplus	brew install --cask tableplus
Browser	sudo apt install ./google-chrome-stable...	brew install --cask google-chrome

3. PHP Environment & Multi-Version Management

3.1 Installing PHP via Homebrew

```
brew install php
brew install php@8.0 php@8.1 php@8.2 php@8.3
```

3.2 Managing PHP Services

To start, stop, or restart PHP services on macOS:

```
brew services start php
brew services stop php@8.0
brew services restart php
```

4. Composer & Laravel Ecosystem

4.1 Global Composer Installation

```
brew install composer
```

4.2 Global Laravel Installer

```
composer global require laravel/installer
```

4.3 Configuring Laravel Valet (macOS Native)

On Linux, cpriego/valet-linux was used. macOS provides native support via standard **Laravel Valet**, which automatically configures DnsMasq and Nginx local proxies.

```
composer global require laravel/valet
valet install
cd ~/Sites
valet park
```

5. Database Setup (MySQL)

5.1 Starting MySQL Service

```
brew services start mysql
```

5.2 Secure Installation & Root User Configuration

```
mysql_secure_installation
```

5.3 Database Creation & Password Authentication

```
mysql -u root -p
CREATE DATABASE my_database;
ALTER USER 'root'@'localhost' IDENTIFIED BY 'your_database_password';
FLUSH PRIVILEGES;
```

6. Version Control (Git & SSH Authentication)

6.1 Global Git Configuration

```
git config --global user.name "your_github_username"
git config --global user.email "your_github_email@example.com"
```

6.2 SSH Key Generation & Keychain Integration

```
ssh-keygen -t rsa -b 4096 -C "your_github_email@example.com"
eval "$(ssh-agent -s)"
```

On macOS, store SSH passphrases in the native Keychain by modifying ~/.ssh/config:

```
Host github.com
  AddKeysToAgent yes
  UseKeychain yes
  IdentityFile ~/.ssh/id_rsa
ssh-add --apple-use-keychain ~/.ssh/id_rsa
```

7. Terminal Customization (Zsh, Oh My Zsh, Powerlevel10k)

macOS uses **Zsh** as its default shell, removing the need to manually switch from Bash.

7.1 Installing Oh My Zsh

```
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install
.sh)"
```

7.2 Installing Powerlevel10k Theme

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git
${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

Set ZSH_THEME="powerlevel10k/powerlevel10k" inside your ~/.zshrc file and run source ~/.zshrc to complete configuration.

8. Troubleshooting & Command Reference

- **Permission Fixes:** In macOS, avoid using chmod 777 on system directories. Keep web project directories under user space (e.g., ~/Sites or ~/Projects).
- **PHP Database Extensions:** PHP database drivers (PDO MySQL, MySQLi) are built into Homebrew PHP installations by default.
- **Ignoring Platform Requirements:** If composer dependencies fail due to local PHP version mismatches during setup:
  ```
  composer install --ignore-platform-reqs
  ```